

# looqbox challenge

Por Filipi Maciel

## Observações

É solicitado para informar o uso de IA caso necessário, abaixo segue os detalhes de onde usei.

- Erros
  - Basicamente, na moda antiga o que dava pra fazer é pesquisar o erro no stackoverflow, forums, documentação e google de modo geral. Mas o ponto é que isso demandava muito tempo e era dividido em duas etapas, uma para entender por que o erro estava acontecendo e outra para procurar uma solução. Aqui entra um uso útil da IA, que mostra rapidamente o que significa o erro e qual seria uma proposta de solução. Além do mais, atualmente o stackoverflow (principal ponto de encontro para buscar soluções) não possui mais a mesma movimentação de antes, a base de usuários caiu, o que me deixa com poucas alternativas.
- Teste SQL Q3
  - Como já faz alguns anos desde que tive que mexer com SQL eu precisei revisar alguns conceitos. As queries 1 e 2 do SQL Test deu pra fazer sozinho, a IA só deixou mais elegante. Mas a terceira query achei desafiadora, e como resolver na mão me forçaria a consumir um bom tempo lendo documentação e livros pra relembrar os conceitos de JOIN, GROUP BY e também para ficar testando, achei mais pertinente solicitar ajuda pra IA de modo que eu pudesse concentrar em entender cada campo da consulta e sua estrutura, entender o motivo que faz a query funcionar e por que ela entrega o resultado esperado.
- Nos cases o uso foi explicativo, com exemplos e sugestões, pois os conceitos eu me recordava mas tinha dificuldade e lembrar como que era feita a implementação, por exemplo, uma função dinâmica basicamente recebe os parâmetros essenciais e monta a query de forma flexível, mas se eu for usar uma lista vazia para juntar os parâmetros eu posso fazer `conditions += [f"PRODUCT_CODE={product_code}"]`, mas existe forma mais elegante de fazer, utilizando `.append` para adicionar um item na lista. Outro exemplo é usar `"PRODUCT_CODE=" + str(product_code)` pra juntar a variável com a string, mas se usar `f"PRODUCT_CODE={product_code}"` fica mais elegante. Ou seja, na medida que eu propunha soluções, eu também solicitava uma revisão pra IA e tipicamente ela sugeria uma solução mais elegante.

## SQL Test

O teste SQL basicamente possui três questões que devemos resolver com consultas. Inicialmente eu optei por não colocar a conexão direta com o banco looqbox-challenge na engine.

As bibliotecas e conexão com o banco estão definidas da seguinte forma:

```
import pandas as pd
import matplotlib.pyplot as plt
from sqlalchemy import create_engine, text
from sqlalchemy import create_engine

# CONEXÃO
USER = "looqbox-challenge"
PASSWORD = "looq-challenge"
HOST = "35.199.115.174"
PORT = 3306

engine = create_engine(
    f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}"
)
```

Q1) What are the 10 most expensive products in the company?

Consulta:

```
query = """
SELECT
    PRODUCT_COD,
    PRODUCT_NAME,
    PRODUCT_VAL
FROM `looqbox-challenge`.data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10
"""

# Executa query
df = pd.read_sql(query, engine)

print(df)
```

Retorno:

|   | PRODUCT_COD | PRODUCT_NAME                                      | PRODUCT_VAL |
|---|-------------|---------------------------------------------------|-------------|
| 0 | 301409      | Whisky Escoces THE MACALLAN Ruby Garrafa 700ml... | 741.99      |
| 1 | 176185      | Whisky Escoces JOHNNIE WALKER Blue Label Garra... | 735.90      |
| 2 | 315481      | Cafeteira Expresso 3 CORACOES Tres Modo Vermelho  | 499.00      |
| 3 | 100280      | Vinho Portugues Tinto Vintage QUINTA DO CRASTO... | 445.90      |
| 4 | 320046      | Escova Dental Eletrica ORAL B D34 Professional... | 399.90      |
| 5 | 190817      | Champagne Rose VEUVE CLICQUOT PONSARDIM Garraf... | 366.90      |
| 6 | 153795      | Champagne Frances Brut Imperial MOET Rose Garr... | 359.90      |
| 7 | 311397      | Conjunto de Panelas Allegra em Inox TRAMONTINA... | 359.00      |
| 8 | 147706      | Whisky Escoces CHIVAS REGAL 18 Anos Garrafa 750ml | 329.90      |
| 9 | 154431      | Champagne Frances Brut Imperial MOET & CHANDON... | 315.90      |

Q2) What sections do the 'BEBIDAS' and 'PADARIA' departments have?

Consulta:

```
query = """
SELECT DISTINCT
    DEP_NAME,
    SECTION_COD,
    SECTION_NAME
FROM `loqbox-challenge`.data_product
WHERE DEP_NAME IN ('BEBIDAS','PADARIA')
ORDER BY DEP_NAME, SECTION_NAME;
"""

df = pd.read_sql(query, engine)

print(df)
```

Retorno:

|   | DEP_NAME | SECTION_COD | SECTION_NAME       |
|---|----------|-------------|--------------------|
| 0 | BEBIDAS  | 4           | BEBIDAS            |
| 1 | BEBIDAS  | 29          | CERVEJAS           |
| 2 | BEBIDAS  | 31          | REFRESCOS          |
| 3 | BEBIDAS  | 30          | VINHOS             |
| 4 | PADARIA  | 8           | DOCES-E-SOBREMESAS |
| 5 | PADARIA  | 27          | GESTANTE           |
| 6 | PADARIA  | 19          | PADARIA            |
| 7 | PADARIA  | 22          | QUEIJOS-E-FRIOS    |

Q3) What was the total sale of products (in \$) of each Business Area in the first quarter of 2019?

Consulta:

```
query = """
SELECT
    c.BUSINESS_NAME,
    c.BUSINESS_CODE,
    ROUND(SUM(s.SALES_VALUE),2) AS TOTAL_SALES
FROM `looqbox-challenge`.data_store_sales s

JOIN `looqbox-challenge`.data_store_cad c
ON s.STORE_CODE = c.STORE_CODE

WHERE s.DATE BETWEEN '2019-01-01'
AND '2019-03-31'

GROUP BY
    c.BUSINESS_NAME,
    c.BUSINESS_CODE

ORDER BY TOTAL_SALES DESC;
"""

df = pd.read_sql(query, engine)

print(df)
```

Retorno:

|   | BUSINESS_NAME | BUSINESS_CODE | TOTAL_SALES |
|---|---------------|---------------|-------------|
| 0 | Farma         | 4             | 81776691.73 |
| 1 | Varejo        | 1             | 81032347.65 |
| 2 | Atacado       | 5             | 80384884.60 |
| 3 | Proximidade   | 2             | 80171122.80 |
| 4 | Posto         | 3             | 32072326.40 |

## Case 1:

No case 1 é solicitado uma função dinâmica, de modo que vamos manter a mesma lógica ou estrutura da consulta mas que seja flexível para adaptar com os parâmetros passados.

O código em si é basicamente uma função no python que estrutura a consulta para ser realizada com base nos parâmetros passados. A ideia é usar uma lista de condições que guarde os filtros (WHERE) da consulta. Então se o usuário passar um código do produto ou de loja, isso é adicionado na lista de condições. Posteriormente a query é montada com junção de strings.

Essa flexibilidade da função dinâmica pode ser uma faca de dois gumes, pois ao realizar uma consulta sem nenhum parâmetro o usuário acaba realizando uma solicitação ao banco que é demorada e retorna mais de dois milhões de linhas, pois na consulta é feito o uso de `SELECT * FROM...` que puxa a tabela inteira, sendo algo custoso em termos de processamento. Isso pode ser prevenido com algumas estratégias, e é sempre interessante assumir esse tipo de comportamento vindo do usuário, portanto, foi adicionado uma condição para prevenir isso que é passar pelo menos um parâmetro.

Uma vez que as condições estão satisfeitas a query será montada. Posteriormente eu explico melhor, mas eu também fiz outras duas modificações, uma para mudar o formato de quantidade para inteiro, e outra para mudar a visualização de valor float para ter duas casas decimais.

## Código:

```
def retrieve_data(engine, product_code=None, store_code=None, date=None):

    # Condições para filtrar a consulta
    conditions=[]

    if not product_code and not store_code and not date:
        raise ValueError(
            "Informe pelo menos um filtro: product_code, store_code ou date."
        )

    if product_code:
        conditions.append(
            f"PRODUCT_CODE={product_code}"
        )

    if store_code:
        conditions.append(
            f"STORE_CODE='{store_code}'"
        )
```

```

if date:
    conditions.append(
        f"DATE BETWEEN '{date[0]}' AND '{date[1]}'"
    )

where=""

if conditions:
    where="WHERE " + " AND ".join(conditions)

# Monta a query
query=f"""
    SELECT *
    FROM `looqbox-challenge`.data_product_sales
    {where}
"""

df = pd.read_sql(query, engine)

# Coloca quantidade de produto como inteiro
df["SALES_QTY"] = df["SALES_QTY"].astype(int)

return df

# EXECUÇÃO
df = retrieve_data(
    engine,
    product_code=18,
    store_code=1,
    date=[
        "2019-01-01",
        "2019-01-31"
    ]
)

# Para visualizar duas casas decimais no valor financeiro
pd.set_option('display.float_format', '{:.2f}'.format)

print(df)

```

Retorno:

|    | STORE_CODE | PRODUCT_CODE | DATE       | SALES_VALUE | SALES_QTY |
|----|------------|--------------|------------|-------------|-----------|
| 0  | 1          | 18           | 2019-01-01 | 708.50      | 65        |
| 1  | 1          | 18           | 2019-01-02 | 1297.10     | 119       |
| 2  | 1          | 18           | 2019-01-03 | 1144.50     | 105       |
| 3  | 1          | 18           | 2019-01-04 | 1090.00     | 100       |
| 4  | 1          | 18           | 2019-01-05 | 893.80      | 82        |
| 5  | 1          | 18           | 2019-01-06 | 741.20      | 68        |
| 6  | 1          | 18           | 2019-01-07 | 654.00      | 60        |
| 7  | 1          | 18           | 2019-01-08 | 741.20      | 68        |
| 8  | 1          | 18           | 2019-01-09 | 1373.40     | 126       |
| 9  | 1          | 18           | 2019-01-10 | 1068.20     | 98        |
| 10 | 1          | 18           | 2019-01-11 | 1057.30     | 97        |
| 11 | 1          | 18           | 2019-01-12 | 806.60      | 74        |
| 12 | 1          | 18           | 2019-01-13 | 686.70      | 63        |
| 13 | 1          | 18           | 2019-01-14 | 697.60      | 64        |
| 14 | 1          | 18           | 2019-01-15 | 763.00      | 70        |
| 15 | 1          | 18           | 2019-01-16 | 1199.00     | 110       |
| 16 | 1          | 18           | 2019-01-17 | 1068.20     | 98        |
| 17 | 1          | 18           | 2019-01-18 | 1057.30     | 97        |
| 18 | 1          | 18           | 2019-01-19 | 795.70      | 73        |
| 19 | 1          | 18           | 2019-01-20 | 697.60      | 64        |
| 20 | 1          | 18           | 2019-01-21 | 675.80      | 62        |
| 21 | 1          | 18           | 2019-01-22 | 806.60      | 74        |
| 22 | 1          | 18           | 2019-01-23 | 1395.20     | 128       |
| 23 | 1          | 18           | 2019-01-24 | 1035.50     | 95        |
| 24 | 1          | 18           | 2019-01-25 | 1057.30     | 97        |
| 25 | 1          | 18           | 2019-01-26 | 850.20      | 78        |
| 26 | 1          | 18           | 2019-01-27 | 763.00      | 70        |
| 27 | 1          | 18           | 2019-01-28 | 708.50      | 65        |
| 28 | 1          | 18           | 2019-01-29 | 730.30      | 67        |
| 29 | 1          | 18           | 2019-01-30 | 1384.30     | 127       |
| 30 | 1          | 18           | 2019-01-31 | 1177.20     | 108       |

Explicação:

O primeiro retorno estava sendo algo assim:

|     | STORE_CODE | PRODUCT_CODE | DATE       | SALES_VALUE | SALES_QTY |
|-----|------------|--------------|------------|-------------|-----------|
| 0   | 1          | 18           | 2019-01-01 | 708.5       | 65.0      |
| 1   | 1          | 18           | 2019-01-02 | 1297.1      | 119.0     |
| 2   | 1          | 18           | 2019-01-03 | 1144.5      | 105.0     |
| ... |            |              |            |             |           |

O que me deixou curioso para entender porquê `SALES_QTY` não estava como inteiro, e valor de vendas em `SALES_VALUE` só tinha uma casa decimal. Depois de investigar percebi que isso é devido a forma como o pandas lê esses valores, pois apesar de em `data_product_sales` estar como decimal para `SALES_VALUE decimal(10,2)` e `SALES_QTY decimal(10,0)`, ao executar `print(df.dtypes)` o retorno dos tipos dessas colunas será `float64`, e o pandas lê e representa como o float e por padrão adiciona uma casa decimal.

Pra deixar a representação mais fiel basta alterar a forma de exibição desses valores: `df["SALES_QTY"] = df["SALES_QTY"].astype(int)` e `pd.set_option('display.float_format', '{:.2f}'.format)`. Isso é relevante pois como mencionado nas informações extras, outras pessoas de outros times vão usar, então uma representação mais clara e precisa ajuda eles a interpretar os valores corretamente.

## Case 2:

No case 2, se tentarmos executar as duas queries vai dar erro, pois não tem database selecionada. A solução pra isso sem fazer outra query e sem modificar as mesmas é alterando a conexão, de modo que já seja estabelecida a conexão com o banco antes de executar as queries. Dessa forma conseguimos obter o resultado sem alterar as queries respeitando a primeira instrução.

Pra cumprir com a segunda instrução é necessário realizar a consulta, trazer o resultado pra um DataFrame e aí sim da pra aplicar o filtro na data. Para tal, usa-se o `.to_datetime()` pra mudar o formato. Também é necessário usar `.between` para especificar um intervalo de data.

## Código:

A conexão tem que alterar pra isso, pois não posso mudar a query:

```
engine = create_engine(
    f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}/loobox-challenge"
)
```

O código fica assim:

```
query1 = """
SELECT STORE_CODE, STORE_NAME, START_DATE, END_DATE, BUSINESS_NAME,
BUSINESS_CODE
FROM data_store_cad
"""

query2 = """
SELECT STORE_CODE, DATE, SALES_VALUE, SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""
```



```

# Realiza consulta no banco e salva em df1 e df2
df1 = pd.read_sql(query1, engine)
df2 = pd.read_sql(query2, engine)

# Passa para formato de datetime
df2["DATE"] = pd.to_datetime(df2["DATE"])

# Filtra intervalo
df2_filtered = df2[df2["DATE"].between("2019-10-01", "2019-12-31")]

print(df1)
print(df2_filtered)

```

Retorno:

Query 1:

|    | STORE_CODE | STORE_NAME     | START_DATE | END_DATE | BUSINESS_NAME | BUSINESS_CODE |
|----|------------|----------------|------------|----------|---------------|---------------|
| 0  | 1          | Sao Paulo      | 2006-10-01 |          | Varejo        | 1             |
| 1  | 2          | Chicago        | 2007-10-01 |          | Varejo        | 1             |
| 2  | 3          | Roma           | 2008-10-01 |          | Varejo        | 1             |
| 3  | 4          | Tokio          | 2009-10-01 |          | Varejo        | 1             |
| 4  | 5          | Paris          | 2019-01-01 |          | Proximidade   | 2             |
| 5  | 6          | Berlin         | 2011-10-01 |          | Proximidade   | 2             |
| 6  | 7          | New York       | 2012-10-01 |          | Proximidade   | 2             |
| 7  | 8          | Belem          | 2013-10-01 |          | Proximidade   | 2             |
| 8  | 9          | London         | 2014-10-01 |          | Farma         | 4             |
| 9  | 10         | Hong Kong      | 2019-01-01 |          | Farma         | 4             |
| 10 | 11         | Rio de Janeiro | 2019-01-01 |          | Farma         | 4             |
| 11 | 12         | Madri          | 2019-01-01 |          | Farma         | 4             |
| 12 | 13         | Dubai          | 2019-01-01 |          | Atacado       | 5             |
| 13 | 14         | Bahia          | 2019-01-01 |          | Atacado       | 5             |
| 14 | 15         | Buenos Aires   | 2019-01-01 |          | Atacado       | 5             |
| 15 | 16         | Salvador       | 2019-01-01 |          | Atacado       | 5             |
| 16 | 17         | Sidney         | 2019-01-01 |          | Posto         | 3             |
| 17 | 18         | Bangkok        | 2019-01-01 |          | Posto         | 3             |
| 18 | 19         | Miami          | 2019-01-01 |          | Posto         | 3             |
| 19 | 20         | Vancouver      | 2019-01-01 |          | Posto         | 3             |

## Query 2:

|      | STORE_CODE | DATE       | SALES_VALUE | SALES_QTY |
|------|------------|------------|-------------|-----------|
| 5460 | 1          | 2019-10-01 | 187601.54   | 12160     |
| 5461 | 10         | 2019-10-01 | 139038.86   | 5223      |
| 5462 | 11         | 2019-10-01 | 252687.35   | 8481      |
| 5463 | 12         | 2019-10-01 | 223973.64   | 7659      |
| 5464 | 13         | 2019-10-01 | 187601.54   | 12160     |
| ...  | ...        | ...        | ...         | ...       |
| 7295 | 5          | 2019-12-31 | 193619.94   | 12607     |
| 7296 | 6          | 2019-12-31 | 193619.94   | 12607     |
| 7297 | 7          | 2019-12-31 | 193619.94   | 12607     |
| 7298 | 8          | 2019-12-31 | 191704.64   | 12500     |
| 7299 | 9          | 2019-12-31 | 167081.21   | 5765      |

## Explicação:

Curiosamente em `data_store_sales` não temos o problema de casas decimais nos campos de valor e quantidade, pois seus tipos retornados são `float64` e `int64` já configurados no banco, o que evita ter que fazer tratativas para exibir num formato melhor.

## Visualização:

Essa parte do case 2 pede para, além de respeitar as instruções, trazer uma visualização em tabela para o resultado, de modo que fique visualmente fácil identificar qual é a loja, sua categoria e o ticket médio. Aqui eu reaproveito o que já foi feito antes com as datas, e agora preciso agrupar com `.groupby` pelo código da loja a soma dos dois valores principais nas colunas com `.agg`, sendo a quantidade e vendas.

Após agrupamento, preciso calcular o ticket médio, que foi definido como:

$$TM = \frac{SALES\_VALUE}{SALES\_QTY}$$

Depois basta juntar os resultados com `merge` e apresentar somente as colunas relevantes. Por fim, para representar a tabela da pra usar o `matplotlib`. Dependendo da situação é mais útil exportar como planilha excel, mas interpretei que era pra fazer algo visual mesmo.

Código:

```
df_store = pd.read_sql(query1, engine)
df_sales = pd.read_sql(query2, engine)

# Muda campo DATE para datetime
df_sales["DATE"] = pd.to_datetime(df_sales["DATE"])

# Filtra data para um intervalo
df_sales = df_sales[df_sales["DATE"].between("2019-10-01",
"2019-12-31")]

# Agrupa por loja
sales_grouped = (
    df_sales
    .groupby("STORE_CODE")
    .agg({ # define o que fazer com cada coluna, aqui é soma
        "SALES_VALUE": "sum",
        "SALES_QTY": "sum"
    })
    .reset_index() # transforma em coluna
)

# Calcula o Ticket Médio
sales_grouped["TM"] =
(sales_grouped["SALES_VALUE"]/sales_grouped["SALES_QTY"])

# Junta com cadastro das lojas
result = sales_grouped.merge(df_store, on="STORE_CODE", how="left")

# Seleciona colunas
result = result[["STORE_NAME", "BUSINESS_NAME", "TM"]]

# Renomeia colunas
result.columns = ["Loja", "Categoria", "TM"]

# Para visualizar duas casas decimais em TM
pd.set_option('display.float_format', '{:.2f}'.format)

print(result)

# GERA TABELA NO MATPLOTLIB

result_display = result.copy() # Copia dataframe
result_display["TM"] = result_display["TM"].map(
```

```

        lambda x: f"{x:.2f}" # Formata para duas casas decimais em TM
    )

fig, ax = plt.subplots(figsize=(8, 6)) # Cria figura 8x6 (polegadas)
ax.axis('off') # Remove eixos

# Cria tabela
table = ax.table(
    cellText=result_display.values, # Valores em cada celula
    colLabels=result_display.columns, # Labels das colunas
    loc='center' # Centraliza a tabela
)

# Configura font, escala...
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.5)

plt.show()

```

Retorno:

|    | Loja           | Categoria   | TM    |
|----|----------------|-------------|-------|
| 0  | Sao Paulo      | Varejo      | 15.39 |
| 1  | Chicago        | Varejo      | 15.53 |
| 2  | Roma           | Varejo      | 15.39 |
| 3  | Tokio          | Varejo      | 15.39 |
| 4  | Paris          | Proximidade | 15.39 |
| 5  | Berlin         | Proximidade | 15.39 |
| 6  | New York       | Proximidade | 15.39 |
| 7  | Belem          | Proximidade | 15.37 |
| 8  | London         | Farma       | 28.99 |
| 9  | Hong Kong      | Farma       | 26.35 |
| 10 | Rio de Janeiro | Farma       | 29.59 |
| 11 | Madri          | Farma       | 29.03 |
| 12 | Dubai          | Atacado     | 15.39 |
| 13 | Bahia          | Atacado     | 15.39 |
| 14 | Buenos Aires   | Atacado     | 15.39 |
| 15 | Salvador       | Atacado     | 15.39 |
| 16 | Sidney         | Posto       | 13.67 |
| 17 | Bangkok        | Posto       | 13.67 |
| 18 | Miami          | Posto       | 13.67 |
| 19 | Vancouver      | Posto       | 13.67 |

| Loja           | Categoria   | TM    |
|----------------|-------------|-------|
| Sao Paulo      | Varejo      | 15.39 |
| Chicago        | Varejo      | 15.53 |
| Roma           | Varejo      | 15.39 |
| Tokio          | Varejo      | 15.39 |
| Paris          | Proximidade | 15.39 |
| Berlin         | Proximidade | 15.39 |
| New York       | Proximidade | 15.39 |
| Belem          | Proximidade | 15.37 |
| London         | Farma       | 28.99 |
| Hong Kong      | Farma       | 26.35 |
| Rio de Janeiro | Farma       | 29.59 |
| Madri          | Farma       | 29.03 |
| Dubai          | Atacado     | 15.39 |
| Bahia          | Atacado     | 15.39 |
| Buenos Aires   | Atacado     | 15.39 |
| Salvador       | Atacado     | 15.39 |
| Sidney         | Posto       | 13.67 |
| Bangkok        | Posto       | 13.67 |
| Miami          | Posto       | 13.67 |
| Vancouver      | Posto       | 13.67 |

### Case 3:

No case 3 tive a ideia de investigar uma correlação entre gêneros dos filmes e sua receita. A ideia era responder a pergunta: “*Existe algum gênero que tipicamente gera mais receita?*”. Para tal, usei a biblioteca do Matplotlib para gerar um gráfico de barras que mostra o total das receitas por gênero. Essa visualização é ideal para quantidades associadas a um conjunto específico de labels (nesse caso o gênero do filme).

### Código:

```
query = """
SELECT
    Genre,
    SUM(RevenueMillions) AS total_revenue
FROM IMDB_movies
WHERE Genre IS NOT NULL
AND RevenueMillions IS NOT NULL
GROUP BY Genre
ORDER BY total_revenue DESC
LIMIT 10;
"""
```

```

df = pd.read_sql(query, engine)

# Tamanho da figura gerada
plt.figure(figsize=(12, 8))

# Define eixos X e Y de um gráfico de barras
plt.barh(df["Genre"], df["total_revenue"])

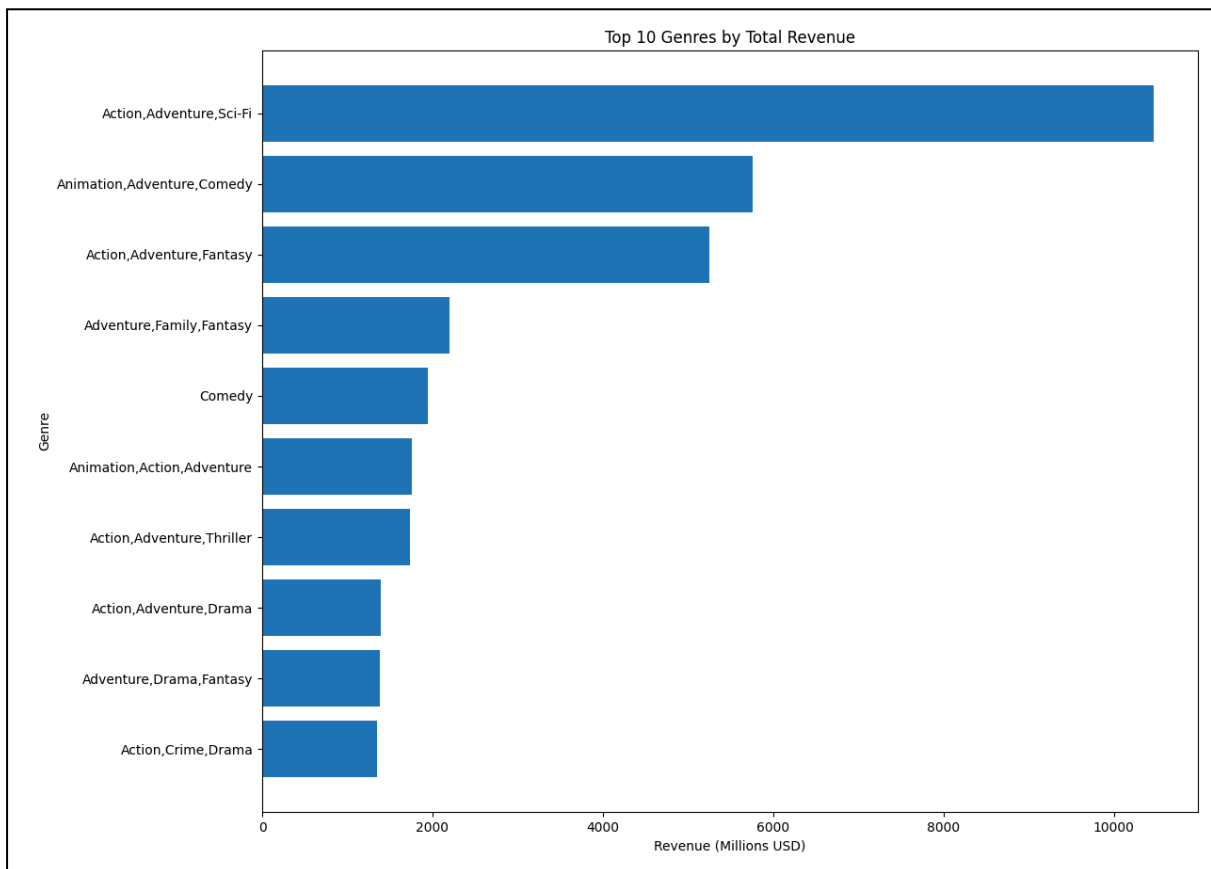
# Títulos e labels
plt.title("Top 10 Genres by Total Revenue")
plt.xlabel("Revenue (Millions USD)")
plt.ylabel("Genre")

# Coloca o maior valor no topo
plt.gca().invert_yaxis()

plt.tight_layout() # Ajusta espaços
plt.savefig("receita_por_genero.png", dpi=300) # Salva arquivo
plt.show()

```

Retorno:



## Explicação:

Como podemos ver, os filmes tipicamente não são definidos por um único gênero, mas sim por uma combinação dos mesmos. Mas ainda assim, todo filme tem um gênero principal que se destaca além dos demais. Percebemos que ação e aventura estão quase sempre em combinação com outros gêneros.

O principal destaque vai para o gênero de ficção científica, com o primeiro lugar, que arrecada quase que o dobro do seu antecessor mais próximo, passando de cem milhões de dólares em receita. Outro destaque interessante está nos filmes do gênero drama, pois estão nos últimos três lugares.

Que nos leva a um contraste interessante, pois comédia, fantasia e ficção estão nos três primeiros, e se formos analisar quais sentimentos esses gêneros despertam nas pessoas veremos um claro contraste em algo alegre, fantasioso, engraçado com algo triste e dramático. Talvez porque sentimentos positivos estão atrelados a liberação de dopamina nas pessoas, e assim elas naturalmente vão se interessar por coisas positivas, que eleva a busca por títulos que despertam esses sentimentos, e assim, fazendo suas respectivas receitas subirem.

## Código Completo:

```
import pandas as pd
from sqlalchemy import create_engine, text

import matplotlib.pyplot as plt
from sqlalchemy import create_engine

### CONEXÃO
USER = "loobox-challenge"
PASSWORD = "loox-challenge"
HOST = "35.199.115.174"
PORT = 3306

engine = create_engine(
    f"mysql+pymysql://{USER}:{PASSWORD}@{HOST}:{PORT}/loobox-challenge"
)

''' SQL TEST - query questão 1

query = """
SELECT
    PRODUCT_COD,
    PRODUCT_NAME,
    PRODUCT_VAL
```

```
FROM `looqbox-challenge`.data_product
ORDER BY PRODUCT_VAL DESC
LIMIT 10
"""
```

```
df = pd.read_sql(query, engine)
```

```
print(df)
'''
```

```
''' SQL TEST - query questão 2
query = """
SELECT DISTINCT
    DEP_NAME,
    SECTION_COD,
    SECTION_NAME
FROM `looqbox-challenge`.data_product
WHERE DEP_NAME IN ('BEBIDAS','PADARIA')
ORDER BY DEP_NAME, SECTION_NAME;
"""
```

```
df = pd.read_sql(query, engine)
```

```
print(df)
'''
```

```
''' SQL TEST - query questão 3
query = """
SELECT
    c.BUSINESS_NAME,
    c.BUSINESS_CODE,
    ROUND(SUM(s.SALES_VALUE),2) AS TOTAL_SALES
FROM `looqbox-challenge`.data_store_sales s

JOIN `looqbox-challenge`.data_store_cad c
ON s.STORE_CODE = c.STORE_CODE

WHERE s.DATE BETWEEN '2019-01-01'
AND '2019-03-31'

GROUP BY
    c.BUSINESS_NAME,
    c.BUSINESS_CODE

ORDER BY TOTAL_SALES DESC;
"""
```



```

df = pd.read_sql(query, engine)

print(df)
'''

''' CASE 1
### FUNÇÃO DINÂMICA

def retrieve_data(engine, product_code=None, store_code=None,
date=None):

    # Condições para filtrar a consulta
    conditions=[]

    if not product_code and not store_code and not date:
        raise ValueError(
            "Informe pelo menos um filtro: product_code, store_code ou
date."
        )

    if product_code:
        conditions.append(
            f"PRODUCT_CODE={product_code}"
        )

    if store_code:
        conditions.append(
            f"STORE_CODE='{store_code}'"
        )

    if date:
        conditions.append(
            f"DATE BETWEEN '{date[0]}' AND '{date[1]}'"
        )

    where=""

    if conditions:
        where="WHERE " + " AND ".join(conditions)

    # Monta a query
    query=f"""
        SELECT *

```

```

        FROM `loobox-challenge`.data_product_sales
        {where}
    """

    df = pd.read_sql(query, engine)

    # Coloca com inteiro pois quantidade de produto nesse caso faz mais
    sentido ser inteiro
    df["SALES_QTY"] = df["SALES_QTY"].astype(int)

    return df

### EXECUÇÃO
df = retrieve_data(
    engine,
    product_code=18,
    store_code=1,
    date=[
        "2019-01-01",
        "2019-01-31"
    ]
)

# Para visualizar duas casas decimais no valor financeiro
pd.set_option('display.float_format', '{:.2f}'.format)

print(df)
'''

''' CASE 2 - PARTE 1
df1 = pd.read_sql(query1, engine)
df2 = pd.read_sql(query2, engine)

df2["DATE"] = pd.to_datetime(df2["DATE"])

df2_filtered = df2[
    df2["DATE"].between("2019-10-01", "2019-12-31")
]

print(df1)
print(df2_filtered)
'''

```

```

''' CASE 2 - PARTE 2
query1 = """
SELECT STORE_CODE, STORE_NAME, START_DATE, END_DATE, BUSINESS_NAME,
BUSINESS_CODE
FROM data_store_cad
"""

query2 = """
SELECT STORE_CODE, DATE, SALES_VALUE, SALES_QTY
FROM data_store_sales
WHERE DATE BETWEEN '2019-01-01' AND '2019-12-31'
"""

df_store = pd.read_sql(query1, engine)
df_sales = pd.read_sql(query2, engine)

# muda campo DATE para datetime
df_sales["DATE"] = pd.to_datetime(df_sales["DATE"])

# Filtra data para um intervalo
df_sales = df_sales[
    df_sales["DATE"].between("2019-10-01", "2019-12-31")
]

# Agrupa por loja
sales_grouped = (
    df_sales
    .groupby("STORE_CODE")
    .agg({ # define o que fazer com cada coluna, aqui é soma
        "SALES_VALUE": "sum",
        "SALES_QTY": "sum"
    })
    .reset_index() # transforma em coluna
)

# Calcula o Ticket Médio
sales_grouped["TM"] = (
    sales_grouped["SALES_VALUE"]
    / sales_grouped["SALES_QTY"]
)

# Junta com cadastro das lojas
result = sales_grouped.merge(
    df_store,
    on="STORE_CODE",
    how="left"
)

```

```

)

# Seleciona colunas
result = result[
    ["STORE_NAME", "BUSINESS_NAME", "TM"]
]

# Renomeia colunas
result.columns = [
    "Loja",
    "Categoria",
    "TM"
]

# Para visualizar duas casas decimais em TM
pd.set_option(
    'display.float_format',
    '{:.2f}'.format
)

print(result)

### GERA TABELA NO MATPLOTLIB

result_display = result.copy() # Copia dataframe
result_display["TM"] = result_display["TM"].map(
    lambda x: f"{x:.2f}" # Formata para duas casas decimais em TM
)

fig, ax = plt.subplots(figsize=(8, 6)) # Cria figura 8x6 (polegadas)
ax.axis('off') # Remove eixos

# Cria tabela
table = ax.table(
    cellText=result_display.values, # Valores em cada célula
    colLabels=result_display.columns, # Labels das colunas
    loc='center' # Centraliza a tabela
)

# Configura font, escala...
table.auto_set_font_size(False)
table.set_fontsize(10)
table.scale(1, 1.5)

plt.show()
'''

```

```

''' CASE 3
query = """
SELECT
    Genre,
    SUM(RevenueMillions) AS total_revenue
FROM IMDB_movies
WHERE Genre IS NOT NULL
AND RevenueMillions IS NOT NULL
GROUP BY Genre
ORDER BY total_revenue DESC
LIMIT 10;
"""

df = pd.read_sql(query, engine)

### GERA GRAFICO NO MATPLOTLIB

# Tamanho da figura gerada em polegadas
plt.figure(figsize=(12, 8))

# Define eixos X e Y de um gráfico de barras
plt.barh(df["Genre"], df["total_revenue"])

# Títulos e labels
plt.title("Top 10 Genres by Total Revenue")
plt.xlabel("Revenue (Millions USD)")
plt.ylabel("Genre")

# Coloca o maior valor no topo
plt.gca().invert_yaxis()

plt.tight_layout() # Ajusta espaços
plt.savefig("receita_por_genero.png", dpi=300) # Salva arquivo, dpi é
qualidade da imagem
plt.show()
'''

```